



ASSET CONTEXT EXPORTER FOR AI



OVERVIEW & BUSINESS VALUE

Asset Context Exporter For AI (AI Context Exporter) is a comprehensive C++ solution designed for solo developers, indie creators, and AAA studios looking to standardize their Data-Driven Pipeline. The tool translates Unreal Engine binary files (`.uasset` , `.umap`) into highly optimized, structured text formats (JSONL). It serves as the ultimate bridge between project architecture and Large Language Models (LLMs) or Agentic AI, empowering both non-programmers and seasoned engineers to build, audit, and scale their projects efficiently.

Secure & Air-Gapped Operation: The tool operates 100% offline within the studio's intranet. There are no external dependencies, third-party APIs, no HTTP request or hidden telemetry. Proprietary code and project logic never leave the local hardware, ensuring strict compliance with corporate Non-Disclosure Agreements (NDAs).

WHAT IS NEW IN V1.2

SINGLE FILE EXPORT

A second export mode sits next to Project Export. Instead of sweeping modules, you browse the project content, queue the individual assets you want and export only those. The browser lists only asset types this tool can actually produce, so a queued asset always yields a file. Single File payloads are written flat in the destination folder, one `.jsonl` per asset, with no module subfolders and no folder hierarchy, and they carry full asset paths instead of the `[*N]` alias tokens, so a single file can be pasted into a chat without the manifest. Output folders are now named after the mode that produced them, `Project_Export_<Time>` or `Single_File_<Time>` , so both modes can share a destination without being confused.

INSTALLS IN ANY PROJECT

Enhanced Input, which ships with Unreal Engine, is now the only required plugin. Niagara, MetaSound and StateTree are declared optional: they are used when the project has them and ignored when it does not. Previously all four had to be enabled just to install the tool.

ANIMATION MODULE REBUILT IN DEPTH

The 1.1 module exported names and bindings but little of the data behind them. 1.2 exports the values an AI needs to reason about a rig:

- > Anim Blueprints reach the same depth as a regular Blueprint: implemented interfaces, real variable types, event dispatchers and the exposed native parent API, on top of the AnimGraphs and State Machines already covered.
- > Skeletons: reference pose with position, rotation AND scale (1.1 exported position only), curve metadata with its flags and linked bones, sockets with full transforms, and blend profiles with the blend scale of every bone.
- > Skeletal Meshes: the full Asset Details panel, including the Post Process Anim Blueprint.
- > IK Rigs and IK Retargeters: solver settings, goals with their transforms, effectors and per-bone stiffness and preferred angles. In 1.1 these were opaque object references.

- Physics Assets: every body with mass, damping, collision channels, axis locks and solver iterations; every collision primitive with dimensions and transform; every constraint with limits, soft settings, projection, break thresholds and the complete drive parameters. This is the difference between identifying a ragdoll and diagnosing one. V1.1 described a full ragdoll as "a few KB"; that compactness was achieved by omitting the values you need to debug one. A character ragdoll now lands around 100-200 KB and is worth reading.
- Anim Sequences and Montages: notifies with trigger time, duration, class and their own properties (1.1 exported only a list of names), sync markers, and the segments of every montage slot with their animation, time range and play rate.

BEHAVIOR TREES WERE EXPORTING ONLY A FRAGMENT OF THE TREE

A Behavior Tree child can be either a task or a nested composite. Only tasks were being walked, so every branch hanging under a nested Selector or Sequence was absent from the export. Measured on Lyra's shooter bot: 2 nodes exported out of 31. Both branches are now walked, and the tree is emitted once instead of being partly duplicated by the generic property dump. Decorators attached to the root of the tree are exported as full nodes with their own properties rather than as bare object references, and where the editor defines an explicit AND / OR / NOT relation between the decorators on a link, that logic is exported alongside them, so an AI can tell a conjunction from a list.

DEPRECATED ENGINE DATA WAS BEING EXPORTED AS IF IT WERE LIVE

Unreal keeps deprecated properties on many structures for backwards compatibility, and UnrealHeaderTool reflects them under their clean name, so a name-based filter cannot see them. They were exported next to the properties that replaced them, holding stale values. The clearest case was collision: a single component reported one collision response table from the deprecated field and a different, contradictory one from the live field. Deprecated properties are now filtered at every level of the walk, including struct members, so each setting appears once, with the value the editor shows.

PROPERTY NAMES ARE NO LONGER USED TO GUESS WHAT TO DROP

1.1 discarded any property whose name contained `"Cached"`, `"Compiled"`, `"GraphNode"` or `"EditorOnly"`. That is a substring match on a name, so it also discarded user data: a variable called `CachedTarget` or an actor flag called `bIsEditorOnlyActor` disappeared with no trace. Worse, the rule was applied inconsistently between the two serializers, so the same property could be present in one part of the export and absent from another, which makes absence meaningless. Filtering is now based on what a property is, not on what it is called.

EACH GRAPH IS EXPORTED ONCE

1.1 wrote every Blueprint and Anim Blueprint graph twice, in two different formats: an ordered execution log and a flattened pseudocode listing. Neither was a superset of the other. They are merged into a single view:

`"ExecutionFlows"` holds the ordered chains starting from each entry point, and `"NodesOutsideExecutionFlow"` holds the nodes no chain reaches, such as orphans, comments and stand-alone pure islands, in the same structured form the chains use. Nodes outside the flow are now described in full instead of as one line of text.

BLUEPRINT COVERAGE GAPS CLOSED

Collapsed sub-graphs, interface implementations and event dispatcher signatures are now exported. Blueprint variables report their concrete type instead of a generic category, so a struct reads as its real name rather than as `"struct"`.

FRAGMENTED MODE NOW SPLITS ASSET PAYLOADS

Fragmented mode capped the manifest index and the source snapshot, but never the asset payloads themselves, so a Max File Size of 512 KB still produced individual asset files of several megabytes. Payloads are now split along their own structure: each part is a valid JSON object carrying the same `"_Path"` plus `"_Part"` and `"_PartCount"`, so the parts can be merged without ambiguity and each one can be read alone. Lists are split element by element, and an element that is itself over the limit is split as well: its pieces carry `"_Item"` with the position that element had in the list, so a reader merges the pieces that share an `"_Item"` value back into one element. Only a value that cannot be divided at all - a single oversized text, for instance - stays whole, and its part is flagged with `"_PartOverSize"`.

EXPORTS ARE REPRODUCIBLE

Some payloads carried references to engine objects that exist only while the editor is running, including their instance number, which changes on every run. Two exports of an unchanged project therefore differed in hundreds of files, which made diffs and caching useless. Such references now export as the object's type rather than its per-run identity. Measured on the reference project: 341 files differed between two consecutive exports before the change, 1 after it, and that one holds a GUID the engine itself regenerates.

MATERIAL LINKS RECORD WHICH OUTPUT AND CHANNEL THEY COME FROM

A material link recorded which node feeds an input but not which of that node's outputs, nor the channel mask on the connection. Roughness fed from a texture's red channel and Roughness fed from its alpha were indistinguishable. Links now carry that information when it is set.

CONTROL RIG EXPORTS ITS PIN VALUES

The RigVM graph used to export node names and connections but not what the nodes were fed. Every pin now carries its value and the graph's wiring is explicit. Because a rig's pin tree is deeper than any other asset's, Control Rig has its own traversal budget instead of raising the global one for all modules.

EMPTY CONFIGURATION IS NOW VISIBLE

A table, list or map left empty used to disappear from the payload entirely, which made "never configured" impossible to tell apart from "not exported". Two rules now apply, and between them they cover every key in a payload:

- > Every container the property walk reaches is written even when it is empty. An empty array or an empty object in the payload means the project has that setting and left it empty.
- > The purpose-built lists that describe what an asset contains are always written too, empty or not: sockets, virtual bones, animation curves, compatible skeletons, notifies, montage slots and sections, physics bodies, collision shapes and constraints, blend space axes, material slots, scalar and vector parameter overrides, input modifiers and triggers, implemented interfaces, event dispatchers, Level Sequence bindings and root tracks. `"Sockets": []` states that the mesh has no sockets; it is an answer, not a gap.

Outside those two rules a key is absent for one reason only: with "Skip default values" on, a value identical to the class default is not written, and a curated view of a node reports only what the author changed. That is the single meaning of absence, and the primer copied by Copy AI Prompt states it, so a model never has to guess whether something was dropped in transit.

One caveat worth knowing: if an asset fails to load a subobject, the entry it would have produced is skipped and the list is written shorter. The export log records the failure, and a payload whose write failed at all is now reported in the validation report rather than passed over in silence.

COPY AI PROMPT RETURNS

Removed in 1.1, back in 1.2 and rewritten. The button copies a short glossary of the notations a model cannot infer on its own: which key is the asset's identity, what an empty container means and what the absence of a key means, the arrows that mark an execution link and a wired input, how a Control Rig sub-graph is addressed, what the [#N] tokens are and how to merge the parts of a split asset. It states no preference about how to answer and assumes nothing about how the payloads reach the model, so it holds up whether a whole export folder is indexed or a single file is dragged into a chat. It is a clipboard action: no file is added to the export.

FASTER EXPORTS

Assets whose type no enabled module can handle are no longer loaded into memory. On Epic's "Valley of the Ancient" this cuts a full export by roughly 40% with an identical number of assets exported. The filter works on asset type, so with a single module enabled the reported count can still exceed what is written: an asset can pass the type filter and belong to a module you left off. The log reports assets found and assets exported separately.

FIXES

- > Two assets sharing a name were appended into the same file. With the default flattened layout this silently merged thousands of unrelated level actors into single files. Output names are now disambiguated deterministically, and actor files are prefixed by their level.
- > Exports to paths longer than the Windows limit, and exports to network drives, failed silently. The extended-length path prefix was being applied in a form the engine's file layer rewrites into an invalid path.
- > The event graph was read from a transient compiler artifact rather than the graph the user authored, so the payload key an AI would look for was often empty.
- > The "Include Asset Metrics" setting had no effect; it now genuinely controls whether referencer counts and memory estimates are collected.
- > Properties declared as fixed-size arrays exported only their first element.
- > Blueprint components were exported twice.
- > Two race conditions in the background disk writer, both on the disk-full path.
- > Headless flags could not force a value opposite to the saved editor settings, so CI runs could not disable Deep World Load or enable the NoAI filter on demand.
- > Anim Sequence and Montage notifies, and montage slot tracks, appeared twice in the same payload: once as a curated view and once inside the generic property dump. The curated view now carries every field of the underlying data and the duplicate is gone.
- > The working asset list the batches share was written into the export folder and left behind if the editor was closed mid-export. It now lives in the project's Intermediate folder, so the export folder holds only exported data.
- > An output folder that cannot be written to - a disconnected network drive, a path the tool is not allowed to create - is now reported as a failure with its cause stated in the process terminal, instead of a bare "invalid parameters" line that blamed a missing output directory when the directory had in fact been given.

- > Pressing Abort in the instant between two batches started one more batch instead of stopping, so the export kept running for minutes after the request.
- > Anim Sequences and Montages carried the full keyframe list of every curve inside the generic property dump, next to the curve summary that replaced it. A single sequence could reach 388 KB of raw keys. Curves now report their name, key count, type flags and comment once, under `"ExtractedCurvesData"`; the per-bone transform curves an animator adds when editing a sequence get the same treatment under `"ExtractedTransformCurvesData"`, so no curve leaves the payload without a trace. Montages gained both summaries, which they never had.
- > Material Functions repeated their expressions: the readable graph, the raw editor-only collection and the engine's dependent expression cache all described the same nodes. Only the readable graph is written now.
- > EQS queries exported their generator but not their tests, because the walk ran out of depth before reaching them: the tests, which decide what the query filters and how it scores, arrived as bare object references. Environment Queries now have their own traversal budget, as Control Rig does.
- > Skeletal meshes with cloth exported the simulation mesh geometry: every vertex position, normal, index and skin weight. The cloth configuration is kept, the coordinate arrays are gone, and every per-vertex array they leave behind reports its count plus, when it holds numbers, the range those numbers span, so a painted mask can still be read at a glance.
- > Behavior Tree and EQS payloads carried the editor's own session history: the graph's debug message log and the last editing viewport position. Those logs named nodes that had been deleted years earlier, which an AI could mistake for part of the query.
- > A Control Rig repeated the same sub-graph once per call site, identically, and every one of them was called `"ContainedGraph"`. Each graph is now written once, carries the `"GraphPath"` that identifies it inside the asset, and repeats are recorded as a reference to the first copy. The node that owns a sub-graph records the path of that sub-graph relative to its own graph, so the link resolves to exactly one entry instead of to a name shared by dozens.
- > Fragmented mode did not descend into an array element that was already over the limit, so a single large graph produced a part well above the requested Max File Size. Such an element is now split too; its pieces carry `"_Item"` with the element's index so they merge back without ambiguity, and `"_PartOverSize"` is left for values that truly cannot be split.
- > Fragmented mode also stopped short whenever the oversized value was a list of one element or an object with a single key, which is the shape of most Blueprint and Behavior Tree payloads: the part was written whole, several times the requested size, and flagged as undividable when it was not. The split now descends through those shapes as well, and the Max File Size is honoured by every file an export writes, the manifest index, the dependency graph and the source snapshot included. A source file longer than the cap is written across consecutive parts, each a closed code block headed with its position.
- > The source snapshot listed files in whatever order the file system returned them, so two exports of an unchanged project produced two different snapshots. The listing is now sorted.
- > A payload that could not be written to disk was logged as a warning and the export still reported success with its self-check passed. Such failures are now counted, reported in the validation report and the export is reported as incomplete.

- > Properties the editor lets you set were dropped by an internal name list: an actor's "Is Spatially Loaded", a component's "Is Editor Only" and "Editable When Inherited", and the actor pivot offset. They are exported again.
- > A Blend Space axis whose name was left at the engine default exported nothing at all, so an Aim Offset that had never been renamed lost its range, its grid divisions and its snapping along with the name. Axes are now selected by how many the asset actually uses, one for a 1D Blend Space and two otherwise, and each carries its name, minimum, maximum, grid divisions, snap-to-grid and wrap-input regardless of what it is called.

INTERFACE

- > The tool window is organised as two tabs, Project Export and Single File, each with its own process terminal and its own resizable layout.
- > The Single File tab splits into folders, content and queue, each column resizable, with the panel heights adjustable as a group.
- > Selecting a module or applying a preset plays a highlight on the modules whose state actually changed.
- > Queueing an asset scrolls the queue to it and highlights the new row; removing one fades it out. When an asset cannot be queued, the reason is printed in the process terminal.

HOW THE EXPORT IS ORGANISED

EXPORT MODES

- > **Standard:** One `.jsonl` file per asset. Ideal for browsing and attaching specific assets.
- > **Fragmented:** One file per asset, split into parts capped at "Max File Size (MB)". Prevents context-window overflows on large payloads.
- > **Consolidated:** One `Master_<Module>.jsonl` per module gathering every asset of that module in a single self-describing JSONL file (one line per asset). An AI can read the entire module context without walking hundreds of files. Individual files are NOT exported in this mode.
- > **Monolithic:** A single project-wide `Master_Project_Context.jsonl` for RAG pipelines, vector databases and autonomous agents.

A single, unambiguous 4-way selector defines the physical layout of the export:

"Preserve Content Drawer Hierarchy" only applies to Standard and Fragmented modes.

FOLDER LAYOUT

Exported folder numbering mirrors the UI module numbering 1:1: `01_Source_Code` , `02_Blueprints` , `03_Data_Assets` , `04_World_Levels` , `05_VFX_Materials` , `06_Animations` , `07_AI_Logic` , `08_Audio` , `09_Input_Mappings` , `10_Physics` (plus `00_Project_Manifest` for the path index, folder tree AND the project dependency graph, which is a project-wide architecture artifact rather than a single blueprint).

DataTables/StringTables/Structs/Enums/GameplayTags live inside `03_Data_Assets` . Skeletons, Skeletal Meshes, Physics Assets and Control Rigs inside `06_Animations` . `10_Physics` holds Physical Materials; the source snapshot and `.ini` configuration share `01_Source_Code` . Output folders are always generated in English.

Inside the directory you choose, exports are organised as `<Project> / <Date> / Project_Export_<Time>` , or `Single_File_<Time>` for the Single File tab. A Single File export writes its payloads flat in that folder and keeps its logs and its validation report in `_Export_Report` .

WORLD LEVELS AND THE EXPORT MODE

Each level respects the chosen export mode. Standard and Fragmented write one file per actor, prefixed with the name of the level it belongs to, plus a per-level `"<Level>__WorldInfo"` file with World Settings and World Partition data. Consolidated and Monolithic gather every actor into the single module or project file. World Levels remains recommended only for RAG or for starting and small maps: this module's token consumption far exceeds every other module combined and will easily consume an entire context window if exported in Consolidated or Monolithic mode.

ANALYTICS DASHBOARD

- > Top tiles: Total Time, Assets Exported, Files on Disk, Total Size (Disk). The physical file count comes from a real disk scan, so Monolithic and Consolidated exports show the true file count while the module chart still breaks down the individual assets.
- > "Export Efficiency" panel: Export Mode, Delta Serialization savings, average size per asset (KB) and physical files written.
 - > "Throughput & Output" panel: assets per second and estimated tokens.
 - > Rankings: the twenty most complex Blueprints, the most referenced assets and the heaviest assets in memory.
 - > The module breakdown counts unique physical files, not individual writes.

TOKEN & METADATA FOOTPRINT

Per-payload metadata is a single `{"_Gen":"YYYY.MM.DD-HH.MM"}` stamp. Bulk numeric arrays are never written: no animation keyframes, no per-bone coordinate dumps, no cloth simulation geometry and no mesh geometry of any kind. What replaces them is a summary that keeps the shape of the data: names, counts, flags and ranges.

MULTI-VERSION ENGINE COMPATIBILITY (UE 5.4 - 5.7)

One single code base compiles across Unreal Engine 5.4, 5.5, 5.6 and 5.7. The plugin detects the engine version at compile time and selects the matching APIs:

- World Partition iteration (ActorDescInstance API, 5.4+).
- SkeletalBodySetup header location (absent on 5.4, PhysicsEngine/ on 5.5+), resolved with `__has_include` guards.
- InstancedStruct location (StructUtils plugin module up to 5.4, CoreUObject on 5.5+), handled both in code and in the Build.cs module dependencies.
- User Defined Structs moved from the Engine module to CoreUObject/StructUtils in UE 5.5, handled by the same compile-time guard.
- Container shrink semantics (EAllowShrinking on 5.4+).

THE EXTRACTION ENGINE & CONTEXT OPTIMIZATION

The core process runs on an asynchronous background worker (a headless commandlet), allowing the Unreal Editor to remain responsive even during massive exports.

- ➤ **Smart Delta Serialization (Simplified to "Skip Default Values")**: evaluates property values against their Class Default Object (CDO). Only variables explicitly modified by the developer are exported, which reduces the size of the export substantially. How much depends entirely on the project.
- ➤ **AST & Visual Logic Translation**: parses uncompiled visual programming, translating execution pins, pure math logic, and variable sets into strictly structured JSON.
- ➤ **Component Architecture Extraction**: reads the CDO and the Simple Construction Script to expose the exact hierarchy and spatial relationships of component templates.
- ➤ **Path Compression**: an aliasing system substitutes deep package paths (/Game/...) with compressed integer identifiers, saving tokens in the LLM's context window.

EXTRACTION MODULES (V1.2)

- **01. Source Code (.cpp / .h)**: raw C++ implementation and engine `.ini` configuration files (optionally including local plugin sources, subject to license validation).
- **02. Blueprints**: uncompiled logic, properties, execution links, component structures, Widget trees/bindings and the project dependency graph.
- **03. Data Assets**: Primary Data Assets, DataTables, CurveTables, StringTables, User Defined Structs, User Defined Enums and the project Gameplay Tag dictionary.
- **04. World Levels**: actor transforms, localized properties, and tags within `.umap` environments (with optional Deep World Partition load), plus Static Mesh metadata (LODs, bounds, material slots, sockets, collision).
- **05. VFX & Materials**: Niagara systems, material graphs (math AST) and material instances.

- **06. Animations:** Skeletons (full reference pose, curve metadata, blend profiles), Skeletal Meshes (full Asset Details including Post Process Anim Blueprint), Physics Assets (every body and constraint value), Anim Blueprints (Blueprint-level depth, state machines included), Blend Spaces, Sequences and Montages (notifies with times and properties, slot segments), Level Sequences, Control Rigs and IK Rigs/Retargeters (solvers, goals, effectors, per-bone settings).
- **07. AI Logic:** Behavior Trees, Blackboards, EQS and StateTrees.
- **08. Audio:** MetaSound graphs and SoundCue structures.
- **09. Input Mappings:** Enhanced Input contexts and actions.
- **10. Physics:** Physical Materials.

QUICK START GUIDE (V1.2)

1. Launch your Unreal Engine project.
2. On the top menu bar, navigate to Tools > Asset Context Exporter For AI.
3. PROJECT EXPORT tab: select a strategic preset (e.g., Core Architecture or Full Project) or toggle modules, pick ONE Export Mode under Export Settings (Standard, Fragmented, Consolidated or Monolithic), define the output directory and click EXPORT PROJECT.
4. SINGLE FILE tab: browse the project content, queue the assets you want, define the output directory and click EXPORT QUEUE. Only those assets are exported, written flat with full asset paths and no alias table.
5. Once finished, use Open Analytics Dashboard to review a project export, or Open Export Folder to attach the payloads to your AI model of choice. Copy AI Prompt puts a primer on the clipboard that tells the model how to read them.

MARKETPLACE COMPLIANCE

The plugin incorporates an active safety mechanism (`bExcludeNoAIAssets` , enabled by default) designed to respect Epic Games FAB "NoAI" directives, automatically bypassing flagged assets during extraction. Disabling this protection transfers all technical and legal responsibility to the user's environment. Activating "Include Plugins" requires an explicit in-editor confirmation acknowledging third-party EULA obligations.

TECHNICAL SPECIFICATIONS (V1.2)

- **Supported Platforms:** Windows 64-bit (Editor-only plugin)
- **Supported Engine Versions:** 5.4, 5.5, 5.6, 5.7 (single code base with compile-time version detection)
- **Required Plugins:** Enhanced Input on 5.5, 5.6 and 5.7; Enhanced Input and StructUtils on 5.4. Both ship with Unreal Engine and the plugin enables them for you. Niagara, MetaSound and StateTree are no longer declared as dependencies: they are used when the project has them and ignored when it does not.
- **Output:** UTF-8 JSONL / Markdown, always English folder & file names
- **Languages:** English only (localized UI may return in a future release)

CONTACT:

mhvan_mx@outlook.com

DOCUMENTATION V1.1 (LEGACY - SUPERSEDED BY V1.2)

Export Modes: replaced the old Monolithic / File Splitting checkboxes with a single 4-way selector (Standard, Fragmented, Consolidated, Monolithic). [STILL CURRENT - see HOW THE EXPORT IS ORGANISED]

Animation System Module (complete redesign of the former "Animations" module): bulk keyframe and spatial arrays dropped in favour of structure; Skeletons (bone tree, sockets, virtual bones, slot groups, animation curves, compatible skeletons, per-bone translation retargeting), Skeletal Meshes, Animation Blueprints (State Machines with states, conduits, transitions and rule ASTs), Blend Spaces, IK Rigs & Retargeters exported via reflection, Control Rig RigVM AST, and Physics Assets moved here from the Physics module. [REBUILT IN DEPTH IN V1.2]

Curve handling: announced as "curves report their key count rather than their keys" and as "curve metadata, structure, not raw keys". [CORRECTED IN V1.2 - the curated summary was written as described, but the generic property dump alongside it still carried every raw key]

Physics Assets: per-body collision dimensions and damping/mass overrides, per-constraint drive parameters, soft limits, projection tolerances and parent-dominance, kept deliberately compact at a few KB. [SUPERSEDED IN V1.2 - compactness was achieved by omitting the values needed to debug a ragdoll]

Data module additions: User Defined Structs, User Defined Enums, and the project Gameplay Tag dictionary exported once as a single `_GameplayTags` file.

Further additions: Static Mesh metadata in the World Levels module; PCG graphs in the Blueprints module, detected without a hard plugin dependency; GAS Ability / Effect / Attribute Set blueprints detected and tagged, their data already exported through the CDO.

UI / folder coherence: exported folder numbering aligned 1:1 with the UI modules. [STILL CURRENT]

World Levels now respect the export mode, instead of always batching actors into consolidated multi-actor files. [STILL CURRENT]

Removed (deliberate simplification): the "01. Project Audits" module and the technical-debt export, whose reports re-stated what developers already know about their own project; the "Copy AI Prompt" button, the `Instructions_for_AI.md` file and the embedded prompt [BUTTON RESTORED IN V1.2, FILE NOT]; "Export UI Visual Bindings" and the "Level of Detail (LOD)" selector, since partial detail levels invited hallucinations and extraction now always runs at full structural depth; multi-language support in 8 languages, the UI remaining localizable but shipping in English only.

Token & metadata optimization: per-payload metadata reduced to a minimal `{"_Gen":"YYYY.MM.DD-HH.MM"}` stamp. [STILL CURRENT]

Analytics dashboard: new top tiles, "Export Efficiency" and "Throughput & Output" panels, physical file count from a real disk scan; Peak RAM usage and every audit-only metric removed. [STILL CURRENT]

Logging overhaul (FAB compliance): the "[ASSET CONTEXT EXPORTER FOR AI]" prefix spam removed; `ContextExporter_Raw.log` line-terminated instead of one endless line; engine version and export mode recorded in the session manifest; an auto-reset event in the background I/O writer, ending a busy-poll loop; the exported folder tree generated at the end of the last batch so it lists every file.

Multi-version engine compatibility (UE 5.4 - 5.7): one code base with compile-time API selection.

[STILL CURRENT]

Fixes: a benign startup "handled ensure" no longer aborts an export with nothing written; multi-megabyte Physics Asset payloads replaced with a dedicated compact exporter; AnimGraph node Details no longer leak editor-only pin-visibility metadata; AnimBP non-animation function graphs no longer silently dropped; the `-NoMaterials` flag is parsed instead of merely emitted; "Animations" no longer requires the "Blueprints" module; FAB "NoAI" compliance enforced by default on fresh installs; World Partition data resolved through the proper engine accessor; the unstable "Data-Driven Rate" metric removed; in Fragmented mode a rename now flushes pending I/O first; path aliasing extended to object references and `_C` class paths; the counting phase persists the exact asset list so batch indices and alias IDs cannot drift; the content filter only skips `/LION` roots instead of any path containing "Localization"; integer `-1 / MAX` values and FName numeric suffixes export verbatim; Widget Blueprint properties no longer exported twice; a failed commandlet launch reports immediately instead of leaving the UI waiting; settings persist to the per-user editor config layer; headless flags `-DeepWorld` , `-MaxRAMMB` and `-NoAIFilter` added for CI parity.

DOCUMENTATION V1.0 (LEGACY - SUPERSEDED BY V1.1)

Overview: the same offline, air-gapped C++ extraction tool, translating `.uasset` and `.umap` into JSONL for LLMs and agentic AI, with Smart Delta Serialization, AST translation of visual logic, component architecture extraction and path compression.

Extraction Modules (V1.0): Source Code (`.cpp` / `.h`), Local Plugins, Blueprints, World Levels, VFX & Materials, Animations, AI Logic, Audio, Data Assets, Input Mappings, Physics.

Architectural Health & Project Audits: Blueprint node complexity scoring, technical debt and orphan asset reports, performance and hierarchy warnings. [REMOVED IN V1.1]

Integrated Multilingual Support: English, Japanese, Simplified Chinese, Korean, German, Spanish, French and Brazilian Portuguese, with the output folder structure always in English. [REMOVED IN V1.1]

Output format: chosen as Monolithic JSONL for vector databases or Fragmented for standard context windows. [REPLACED IN V1.1 BY THE 4-WAY EXPORT MODE SELECTOR]

Technical Specifications (V1.0): Windows 64-bit; supported engine versions 5.5, 5.6, 5.7.

M H V A N

